Guía de Actividades Práctico-Experimentales Nro. 003

1. Datos Generales

Estudiantes	Christopher Alexander Pineda Rodas Eduardo José soto Cabrera Lenin Fabricio Macas Cabrera Willan Sebastian Granda Camacho Jeancarlos Fernando Aguirre Romero
Asignatura	Estructura de Datos
Ciclo	3ro A / B
Unidad	Estructuras de datos estáticas (arreglos, conjuntos, registros, archivos) y estructuras de datos dinámicas (listas, pilas y colas)
Resultado de aprendizaje de la unidad	R1. Implementa algoritmos simples de búsqueda y explica las diferencias en el orden de complejidad computacional, bajo los principios de solidaridad,
Práctica Nro.	03
Título de la Práctica	Mapeo de referencias en memoria y la auditoría de fugas de memoria.
Nombre del Docente	Cristian Ramiro Narváez Guillén
Fecha	jueves 15 y viernes 16 de octubre 2026
Horario	Jueves 15 de 08:30 – 10:30 Viernes 16 de 11h30 – 13h30
Lugar	EVA Aula
Tiempo planificado en el Silabo	3 horas

2. Objetivo(s) de la Práctica:

- Modelar el ciclo de vida de los objetos en memoria, diferenciando físicamente la asignación estática (Stack) de la asignación dinámica (Heap) dentro del motor V8 de JavaScript.
- Auditar y cuantificar empíricamente el desperdicio de memoria (fugas o Memory Leaks) originado por referencias colgantes y cierres léxicos mal gestionados.
- Aplicar principios de Green Computing induciendo manualmente el Recolector de Basura (Garbage Collector) para medir el ahorro de memoria RAM y el decrecimiento de la complejidad espacial $S(n)$ a lo largo del tiempo.

3. Materiales y reactivos:

- Software Node.js (versión LTS 20.x o superior) instalado en el sistema.
- IDE moderno para desarrollo (Visual Studio Code, WebStorm).
- Conexión a Internet estable para investigación en documentación oficial de V8.
- Acceso a la consola del sistema (Terminal de Linux/macOS o PowerShell en Windows).

4. Equipos y herramientas

Recursos Físicos

- Aula de Computación.

Computadora personal o de laboratorio:

- Sistema operativo: Windows, Linux o macOS.
- Procesador: Mínimo 2 núcleos.
- Memoria RAM: Mínimo 4 GB.

5. Procedimiento / Metodología

Enfoque PID-2025:

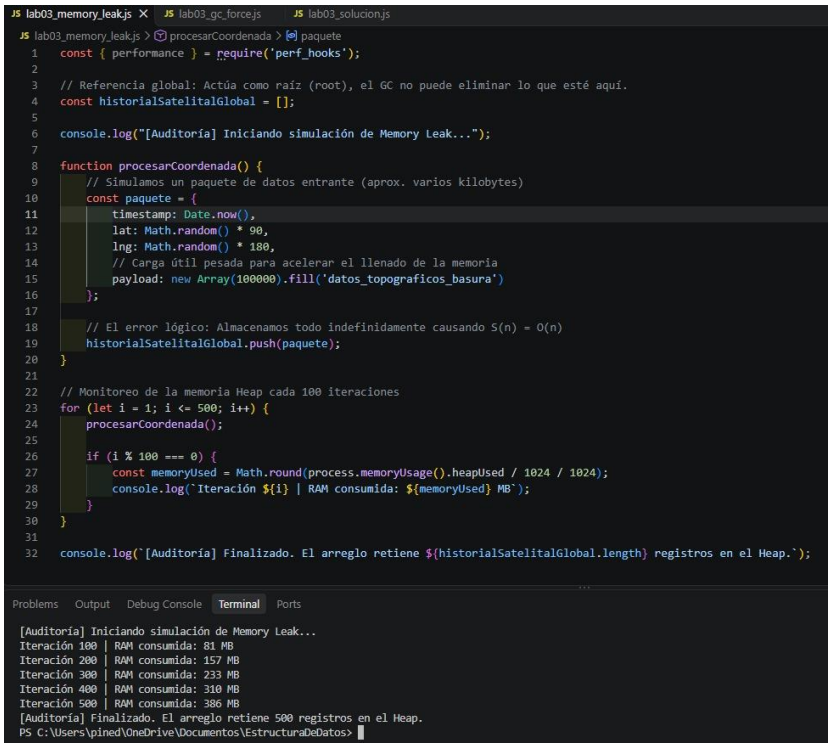
- **ABP (Aprendizaje Basado en Problemas):** El servidor satelital desarrollado en la Práctica 001 lleva funcionando 48 horas continuas procesando coordenadas, pero repentinamente colapsa mostrando el error FATAL ERROR: Ineffective mark-compacts near heap limit Allocation failed - JavaScript heap out of memory. El equipo debe auditar el código, encontrar por qué la memoria crece a un ritmo de $O(n)$ sin límite, y aplicar una solución arquitectónica para estabilizar el consumo.
- **ABI (Aprendizaje Basado en Investigación):** El estudiante investigará cómo opera el algoritmo *Mark-and-Sweep* (Marcado y Barrido) del *Garbage Collector* (GC) y por qué las referencias globales impiden su ejecución.

- **Modalidad:** Individual.

Tarea 1: Simulación de una Fuga de Memoria (Memory Leak)

Para poder auditar un problema de memoria, primero debemos crearlo modelando un historial de ubicaciones satelitales que jamás se descarta.

1. **En el directorio de su proyecto, cree un archivo llamado lab03_memory_leak.js.**
2. **Implemente el siguiente código que simula la llegada asíncrona de datos que se acumulan en el entorno global (Heap).**



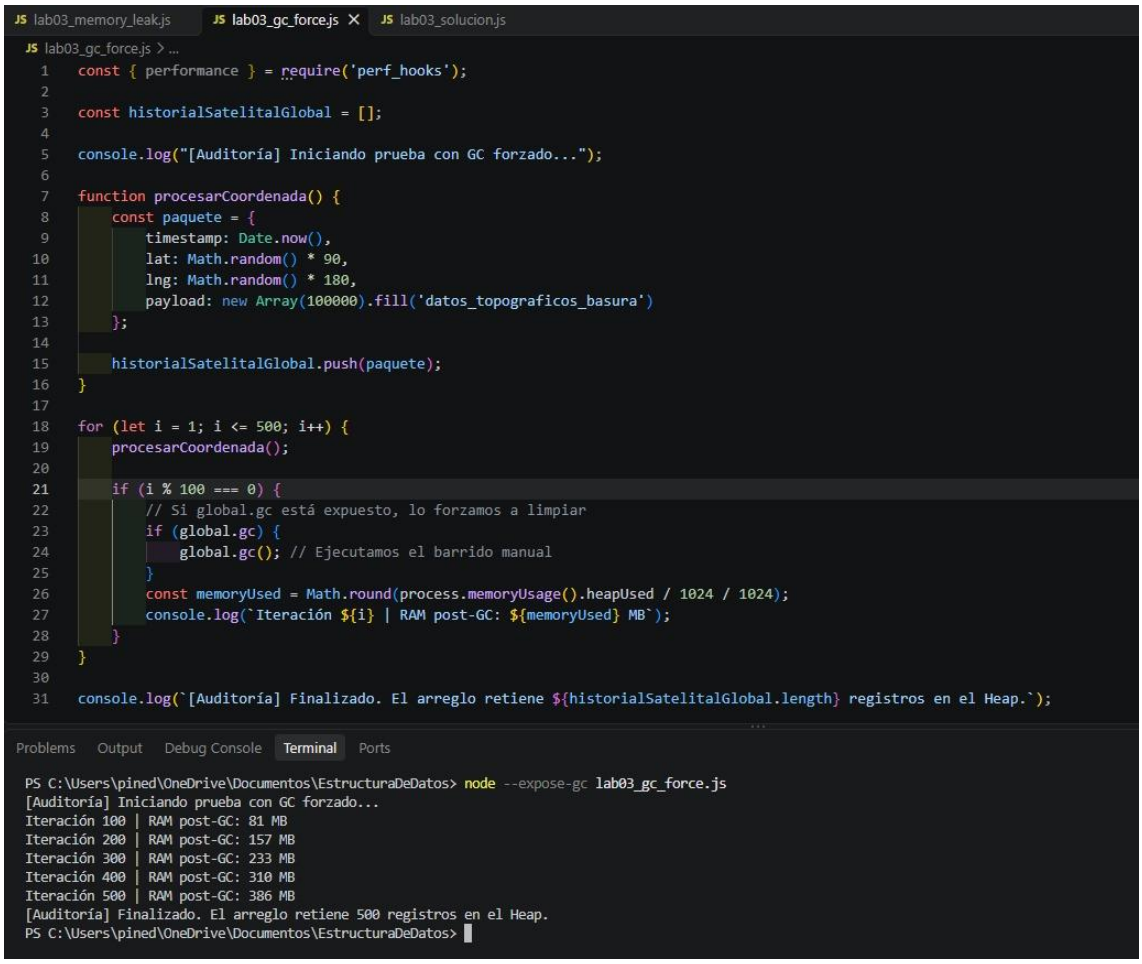
3. Ejecute el script con node lab03_memory_leak.js y observe cómo la RAM crece linealmente.

Tarea 2: Forzado del Recolector de Basura (Garbage Collector)

Por defecto, el GC de Node.js es no determinista. Para medir si estamos solucionando la fuga, forzaremos su ejecución.

1. **Duplique el archivo anterior y nómbrelo lab03_gc_force.js.**
2. **Modifique el ciclo for para invocar al Recolector de Basura. Node.js requiere que esto se haga mediante una bandera especial de ejecución.**

3. **Ejecución estricta: Para exponer la función gc(), ejecute en terminal: node --expose-gc lab03_gc_force.js.**
4. **Análisis: Notará que la RAM sigue creciendo. El *Garbage Collector* no eliminó la basura porque el arreglo global historialSatelitalGlobal mantiene una referencia activa hacia los objetos. El algoritmo *Mark-and-Sweep* los marca como "en uso".**



Tarea 3: Resolución y Mantenimiento Ecológico (O(1) Espacial)

1. **Cree el archivo lab03_solucion.js.**
2. **Refactorice la lógica eliminando referencias obsoletas, permitiendo que el GC las reclame, manteniendo un historial cíclico de máximo 50 registros espaciales.**
3. **Ejecute: node --expose-gc lab03_solucion.js. Compruebe cómo el consumo de memoria alcanza un techo y se mantiene estable, resolviendo el colapso del servidor.**

```
JS lab03_memory_leak.js   JS lab03_gc_force.js   JS lab03_solucion.js
JS lab03_gc_force.js > ...
1  const { performance } = require('perf_hooks');
2
3  const historialSatelitalGlobal = [];
4
5  console.log("[Auditoría] Iniciando prueba con GC forzado...");
6
7  function procesarCoordenada() {
8      const paquete = {
9          timestamp: Date.now(),
10         lat: Math.random() * 90,
11         lng: Math.random() * 180,
12         payload: new Array(100000).fill('datos_topograficos_basura')
13     };
14
15     historialSatelitalGlobal.push(paquete);
16 }
17
18 for (let i = 1; i <= 500; i++) {
19     procesarCoordenada();
20
21     if (i % 100 === 0) {
22         // Si global.gc está expuesto, lo forzamos a limpiar
23         if (global.gc) {
24             global.gc(); // Ejecutamos el barrido manual
25         }
26         const memoryUsed = Math.round(process.memoryUsage().heapUsed / 1024 / 1024);
27         console.log(`Iteración ${i} | RAM post-GC: ${memoryUsed} MB`);
28     }
29 }
30
31 console.log(`[Auditoría] Finalizado. El arreglo retiene ${historialSatelitalGlobal.length} registros en el Heap.`);

Problems  Output  Debug Console  Terminal  Ports

PS C:\Users\pined\OneDrive\Documentos\EstructuraDeDatos> node --expose-gc lab03_gc_force.js
[Auditoría] Iniciando prueba con GC forzado...
Iteración 100 | RAM post-GC: 81 MB
Iteración 200 | RAM post-GC: 157 MB
Iteración 300 | RAM post-GC: 233 MB
Iteración 400 | RAM post-GC: 310 MB
Iteración 500 | RAM post-GC: 386 MB
[Auditoría] Finalizado. El arreglo retiene 500 registros en el Heap.
PS C:\Users\pined\OneDrive\Documentos\EstructuraDeDatos>
```

6. Resultados esperados:

- Los estudiantes lograrán observar de manera tangible cómo una referencia activa en una estructura de datos (Arreglo) impide la liberación de memoria dinámica (*Heap*). Aplicando un límite lógico, lograrán transformar un consumo de complejidad $S(n) = O(n)$ (infinito en el tiempo) a uno constante $S(n) = O(1)$, aplicando principios de diseño energéticamente eficientes.

7. Preguntas de Control:

Desde una perspectiva filosófica profunda y analítica, responda en su informe:

- **Causa Material: Al declarar un objeto complejo dentro de una función, ¿en qué "lugar material" (Stack o Heap) se asienta su estructura principal, y en qué lugar se asienta el puntero que lo referencia?**

  Los objetos suelen almacenarse en el Heap, mientras que las variables locales y sus referencias pueden estar en el Stack, según la implementación del motor de JavaScript.

- **Causa Formal: El contexto global de JavaScript tiene una "forma" que abarca todo el ciclo de vida del programa. ¿De qué manera la estructura formal del alcance (*scope*) global actúa como una jaula que le prohíbe actuar al Recolector de Basura?**

  El alcance global puede mantener referencias a objetos durante toda la ejecución del programa, evitando que el recolector de basura los considere inaccesibles.

- **Causa Eficiente: El algoritmo de *Mark-and-Sweep* actúa como un destructor activo. Sin embargo, en un *Memory Leak*, ¿quién o qué es el agente responsable (causa eficiente) que detiene la hoja de la guadaña del Recolector de Basura?**

  El responsable del memory leak es el código que mantiene referencias innecesarias a los objetos, impidiendo su liberación.

- **Causa Final: Si el hardware permite almacenar Gigabytes de información temporal, ¿cuál es el propósito final (telos) de programar una destrucción intencional de los datos que nosotros mismos hemos creado con tanto esfuerzo computacional?**

  Liberar memoria permite aprovechar mejor los recursos, evitar el consumo excesivo de RAM y mantener el rendimiento del programa.

- **Esencia y Existencia: Si un paquete de coordenadas queda "olvidado" en el último índice de un arreglo sin que ninguna función lo vuelva a consultar jamás, físicamente ocupa electrones en la RAM, pero lógicamente es inaccesible. ¿Existe este dato en la realidad del software o ha perdido su esencia al perder su utilidad? Argumente.**

  Un dato puede seguir ocupando memoria, aunque haya perdido su utilidad. Si ya no es accesible desde las referencias activas del programa, el recolector de basura puede eliminarlo.

8. Evaluación.

Criterio Evaluado	**Puntaje Máximo**	**Descripción**
Desarrollo y Código	5 puntos	Implementación correcta de los scripts y resolución del problema.

Análisis Teórico	3 puntos	Respuestas claras y fundamentadas a las preguntas de control.
Buenas Prácticas	2 puntos	Código ordenado, estructurado y bien documentado.
Total	10 puntos	Puntaje total posible.

9. Bibliografía

- P. W. Bible and L. Moser, *An Open Guide to Data Structures and Algorithms*. PALNI Open Press, 2023. .Available: https://open.palni.org/
- Mozilla Developer Network, "Memory management and Garbage Collection in JavaScript", *MDN Web Docs*, 2023. Available: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Memory_Management
- Node.js Foundation, "V8 JavaScript engine and CLI options", *Node.js API*, 2023. Available: https://nodejs.org/docs/latest-v20.x/api/cli.html

10. Elaboración y Aprobación

- Elaborado por: nombre del Docente responsable de la práctica.
- Aprobado por: nombre del Director de Carrera.

Elaborado por	Cristian R. Narváez Guillén **Docente**	
Aprobado por	Edison L Coronel Romero **Director de Carrera**	